RTOS (Real-Time Operating S/m)

Real-time s/m must ensure a Response within a given time frame or meet a deadline under real-time conditions.

Types :-

Real-time s/ms
- Hard real-time s/m
  - Negligible degree of tolerance for missed deadliness.
  - A missed deadline can lead to catastrophic failure of the s/m.
- Soft real time s/m
  - Deadlines may be missed occasionally, but the s/m doesn't fail.
  - s/m quality is acceptable

RTOS allows embedded s/m to have very precise timing & a high degree of reliability

Features :-

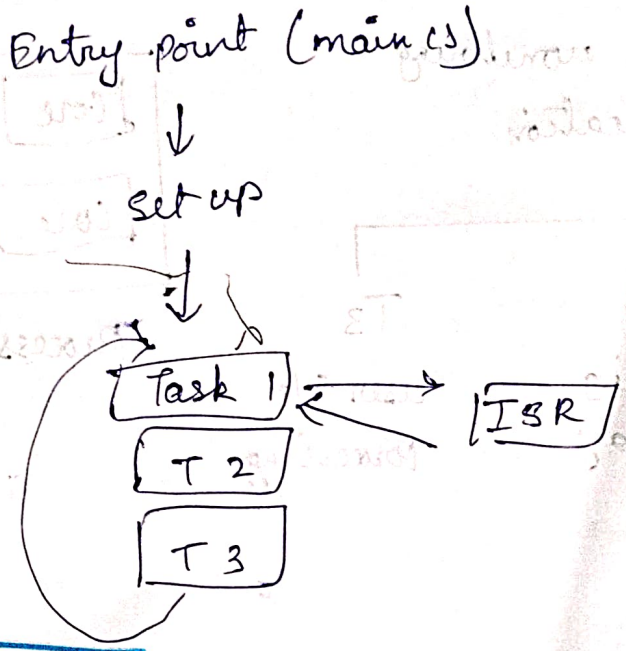
* multitasking
&
* Low Interrupt latency
* Kernel Preemption
* scheduling mechanism

Single-Tasking Vs Multitasking

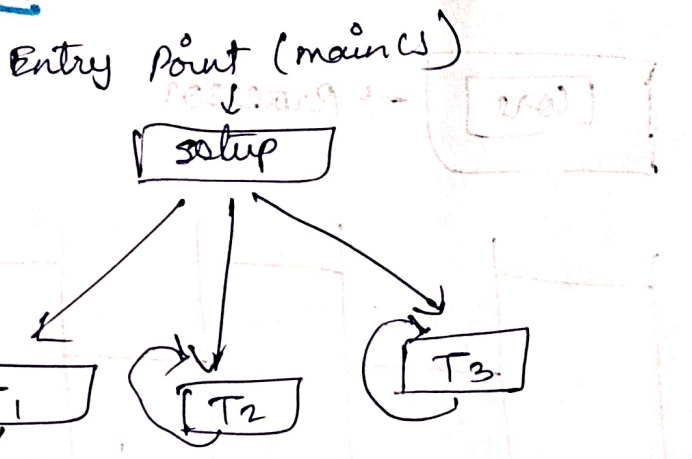
Single Tasking → without RTOS, CPU can execute only one task at a time

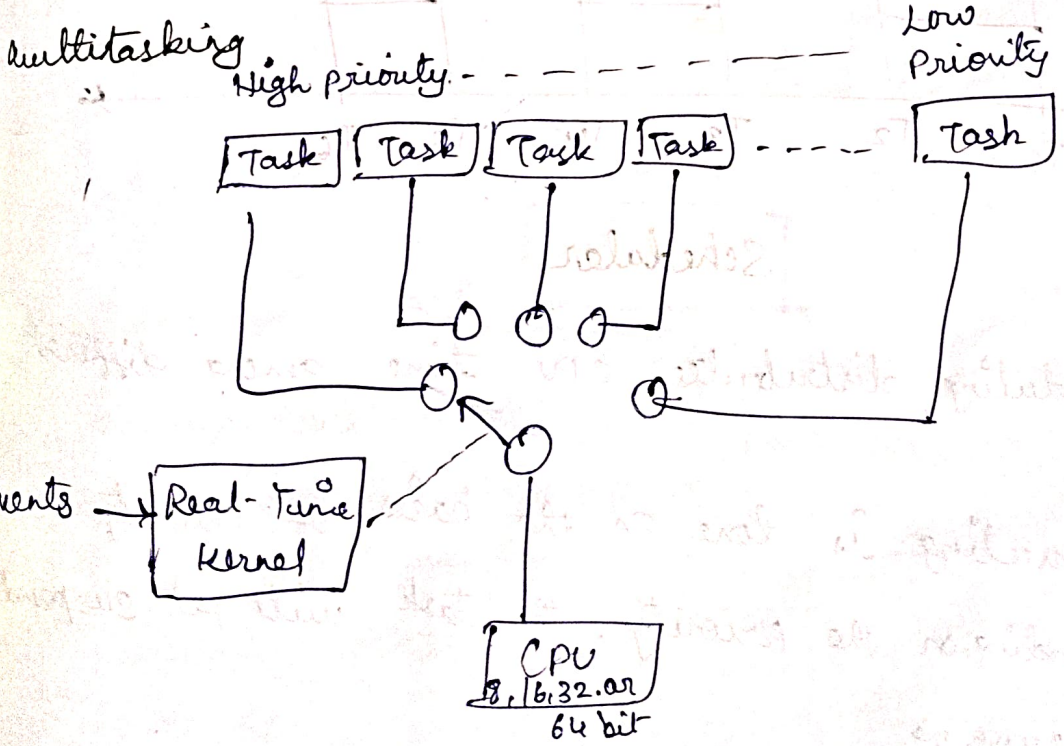
multi → RTOS allows the execution of multiple task on a single CPU multitasking is done by scheduler in RTOS

Single Task S/m

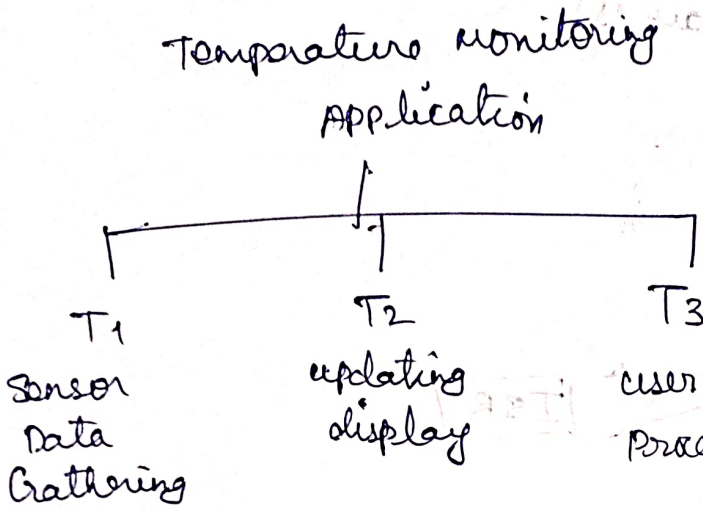


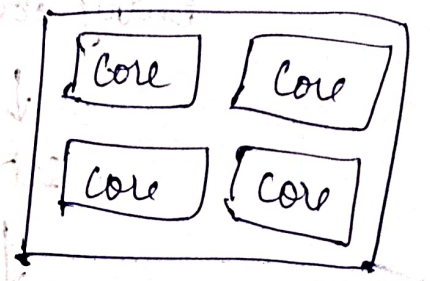
Multi task S/m





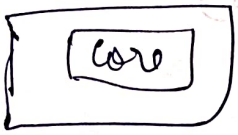
multitasking in different core CPU



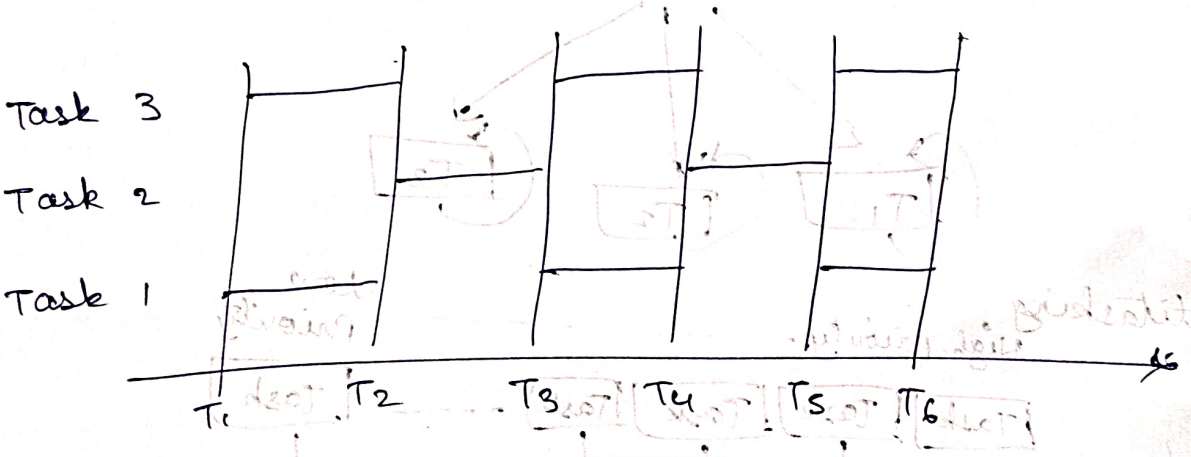


Processor with 4 Cores

Single Core



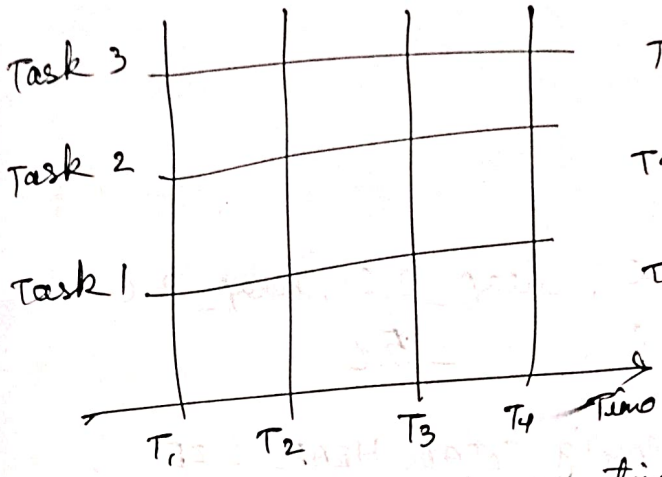
→ Processor



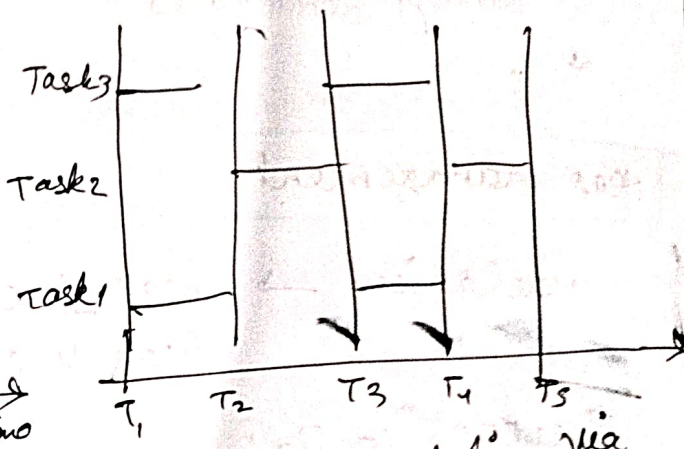
Scheduler

* Scheduling distributes CPU time among different tasks
* scheduling is done on the basis of priority
* Depending on the priority, the task will get suspended or start running.

Multitasking in different core CPU

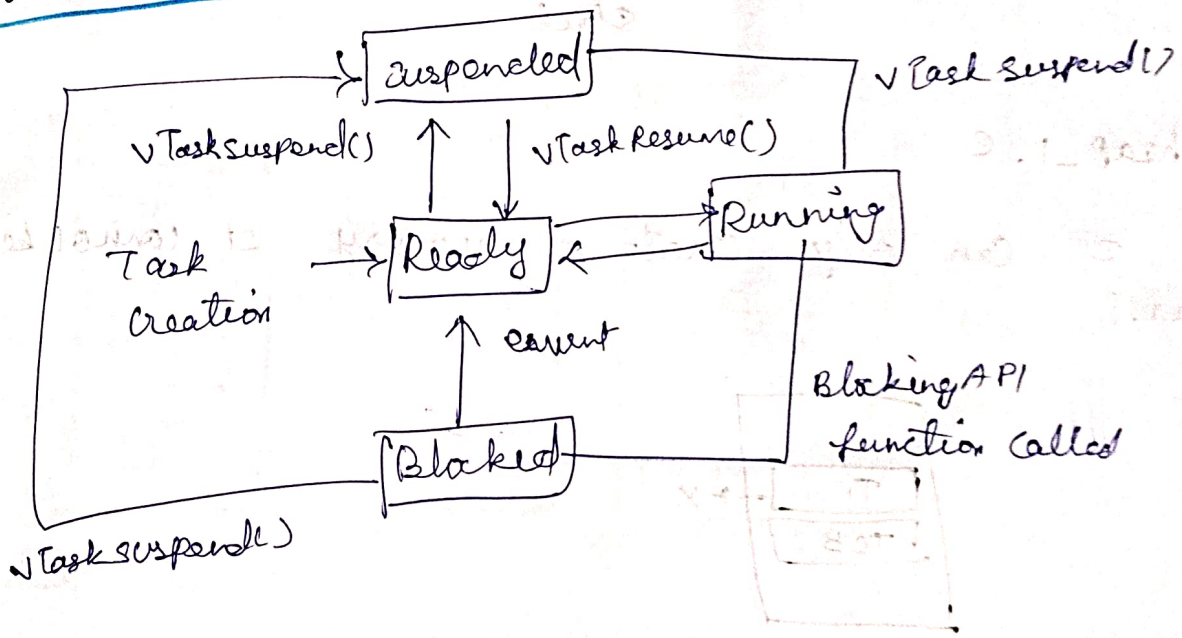


All tasks appear to be executing



task scheduling via Scheduler

Tasks in FreeRTOS state



Features of FreeRTOS

* Tiny, Power-Saving kernel
* Support 40+ architectures
* One-Code base MCU "
* modular libraries
* Add on "
* Pre-emptive or co-operative operation
* Very flexible task priority assignment
* Efficient task notification mechanism

* Queue
* Binary Semaphores
* :

Heap management

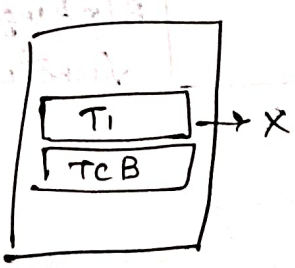
Mechanism → heap_1.c, heap_2.c, heap_3.c
heap_4.c, " _5.c

FreeRTOS config.h parameters → * config TOTAL_HEAP_SIZE
* " SUPPORT_DYNAMIC_ALLOCATION

Resources for Task → Task control Block (TCB)
stack

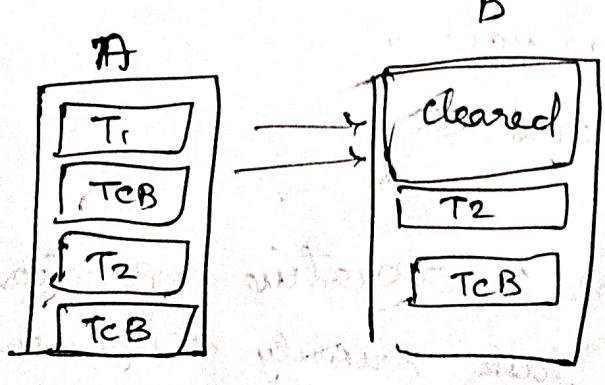
1) heap_1.c

It can only create a memory. It cannot be clear.



2) heap_2.c

To over the heap_1.c we use heap_2.c to clear the memory space. but it can only allocate



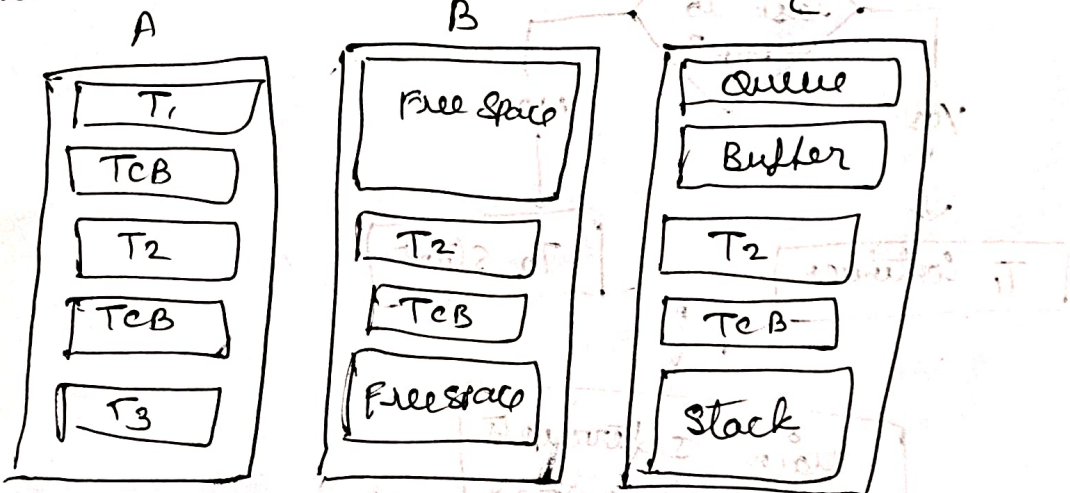
3) heap_3.C

~~In heap_2.c we only allocate the space~~

* make use of Std library functions
* Size of the heap is defined by linker configuration
* Config TOTAL_HEAP_SIZE has no effect
* heap_3 does make malloc() & free() thread_safe.

4) heap_4.C

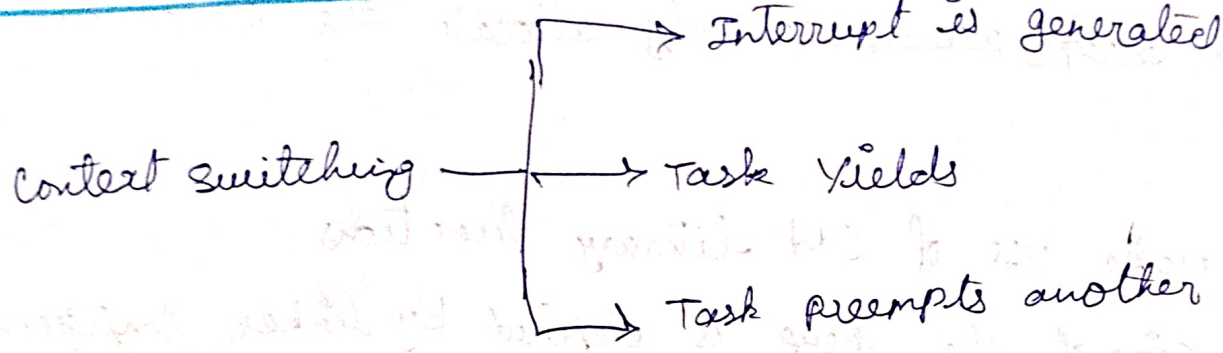
To overcome the heap_2.c we use heap_4.C To allocate the memory as we want

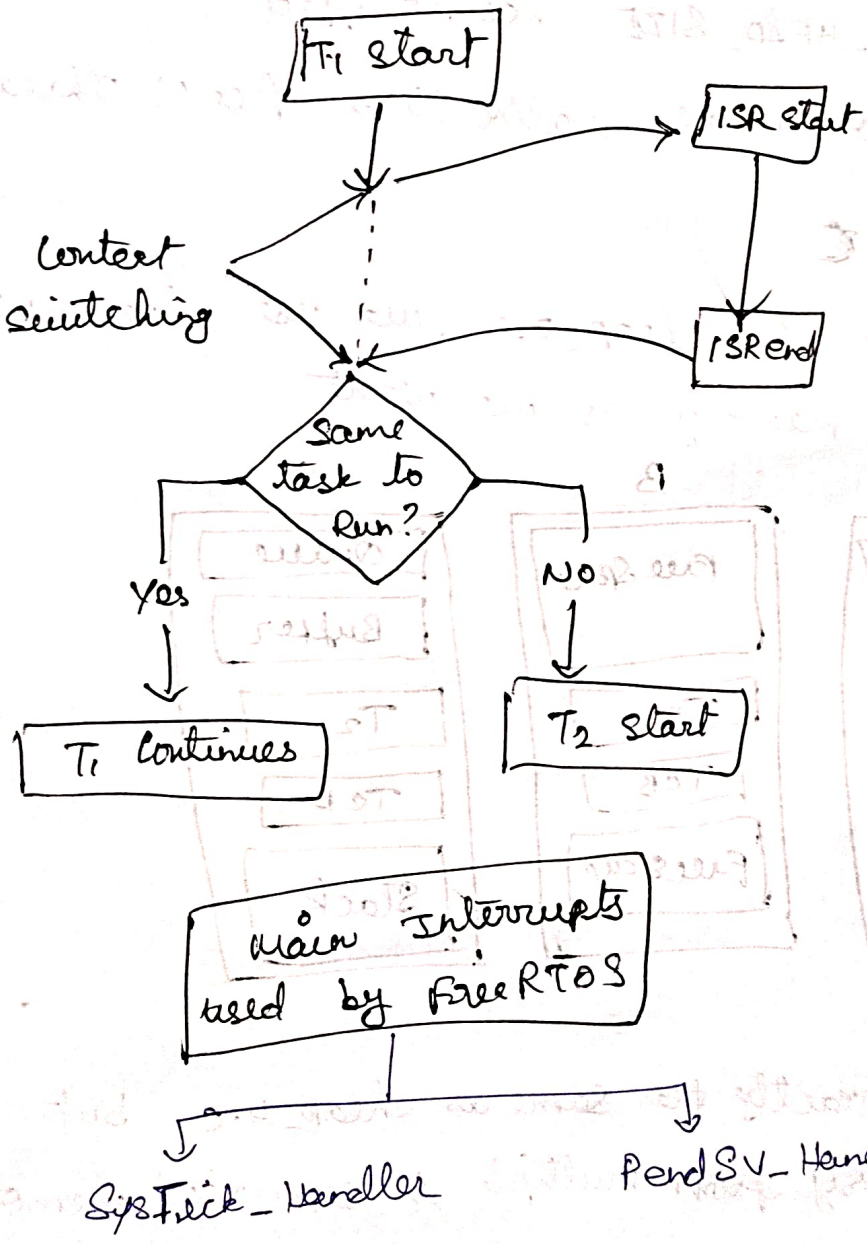


5) heap_5.C

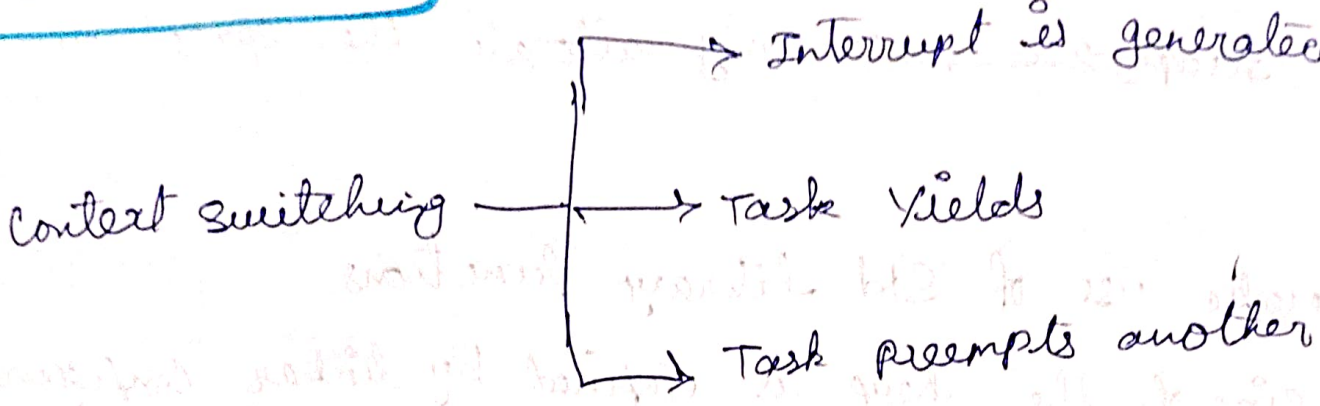
* It is exactly the same as heap_4.C but allows to allocate memory from multiple & separated memory spaces
* In few controllers RAM is not arranged as a single contiguous memory.
* when heap_5.C is used vPortDefineHeapRegions() must be called before any kernel objects

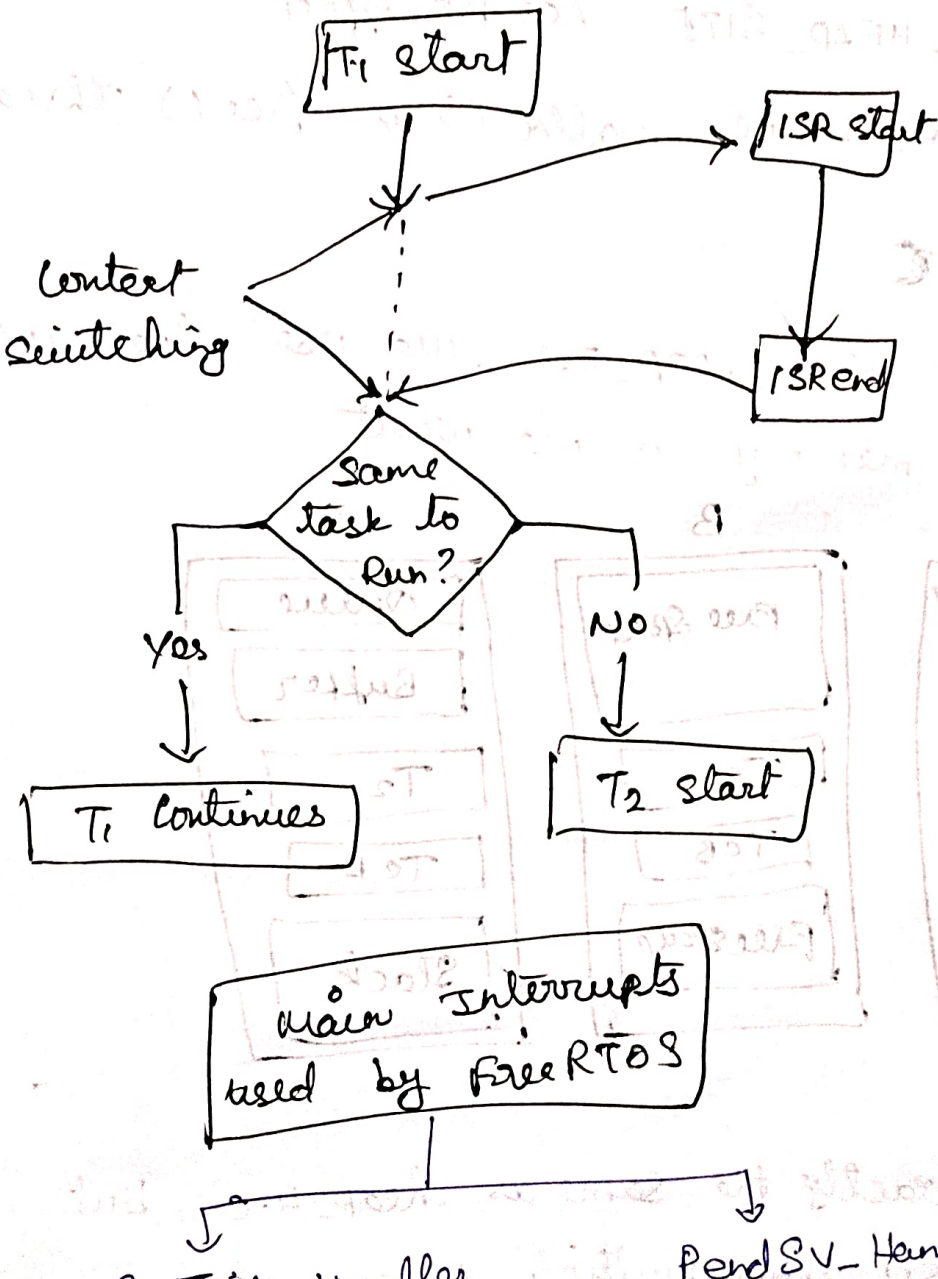
Context switching





Context switching





Request for context switching → Control enters xPortPendSVHandler

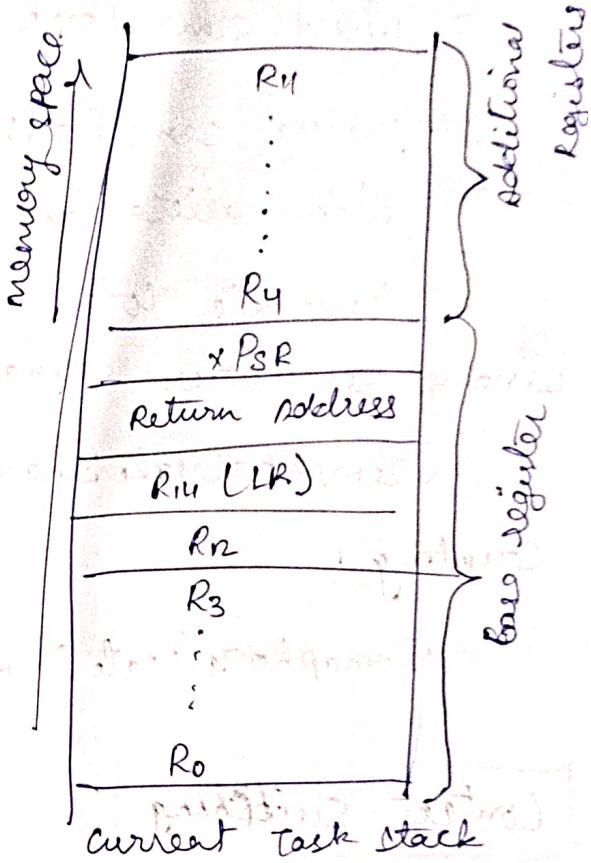
R1, R0 → state of stack is started

R3 → Store the TCB address

R4 .... R11 → Additional registers

R2 → It loads the address of current process

R14 → Link Register



Semaphores in RTOS

Signalling b/w ISRs & Tasks

* Binary Vs Counting semaphores
* Create them in cubeMx
* Give & take from tasks & ISRs

Binary Vs counting Semaphores

Binary → value 0 or 1. signalling : one side gives. the other takes. Perfect for ISR to task event

Counting → value 0 to N. Counts event or guards N identical resources, e.g. 3 DMA buffers.

No ownership → any task or ISR may give. A semaphore is a permission slip, not a lock.

Create semaphores in cube MX

1) Timers & semaphores tab - inside the FREERTOS configuration
2) Binary semaphores → Add - Name it BtnSem
3) Counting semphores → " - " parksem, count 3
4) Pick . allocation
5) Generate code

Binary semphores signalling :-

xSemphorCreate Binary(); // starts EMPTY

Counting :

xSemaphore Create Counting (3, 3);

Context switching

Preemptive - A higher priority ready task . tasks the CPU immediately. Config USE_PREEMPTON = 1

Cooperative - A task runs until it blocks or calls taskYIEL() . one hog freezes everything . config USE_PREEMPTION = 0

Time slicing - Equal priority tasks share the CPU round-Robin , one tick each. Config USE_TIME_SLICING = 1

Essential Task control APIs

API	what it does	state chang
1) vTaskDelay (n)	sleep n ticks from now	Running → Block
2) vTaskDelay until ()	Fixed Period, No drift	Running → "
3) vTaskSuspend (h)	Freeze until resumed	Any → suspen
4) vTaskResume (h)	wake a suspended task	Suspend → Rea
5) taskYIELD ()	offer CPU to equal peers	Running → Rea
6) vTaskDelete (h)	Remove task & free it	Any → Delete

Create semaphores in cube MX

1) Timers & semaphores tab - inside the FREERTOS configuration
2) Binary semaphores → Add - Name it BinSem
3) Counting semphores → " - " parkSem, Count 3
4) Pick . allocation
5) Generate code

Binary semphores signalling :-

xSemphorCreate Binary(); // starts EMPTY

Counting :

xSemaphore Create Counting (3, 3);

Context switching

Preemptive - A higher priority ready task . tasks the CPU immediately. Config USE_PREEMPTON = 1

Cooperative - A task runs until it blocks or calls taskYIE (). one hog freezes everything . config USE_PREEMPTION = 0

Time slicing - Equal priority tasks share the CPU round-Robin . one tick each. Config USE_TIME_SLICING = 1

Essential Task Control APIs

API	what it does	state chang
1) vTaskDelay (n)	sleep n ticks from now	Running → Blo
2) vTaskDelay until ()	Fixed Period, No drift	Running → "
3) vTaskSuspend (h)	Freeze until resumed	Any → Susp
4) vTaskResume (h)	wake a suspended task	Suspend → R
5) taskYIELD ()	offer CPU to equal peers	Running → Re
6) vTaskDelete (h)	Remove task & free it	Any → Delet

Feature	Mutex	Semaphore
1) Purpose	mutual exclusion	signalling / counting
2) Ownership	yes: taker must give	NO: anyone may give
3) Priority inheritance	yes	NO
4) ask inside an ISR	NO	yes (from ISR)
5) Typical value	1 = free, 0 = locked	0 to N
6) Analog	Restroom key	Doorbell, car park

Priority Inversion

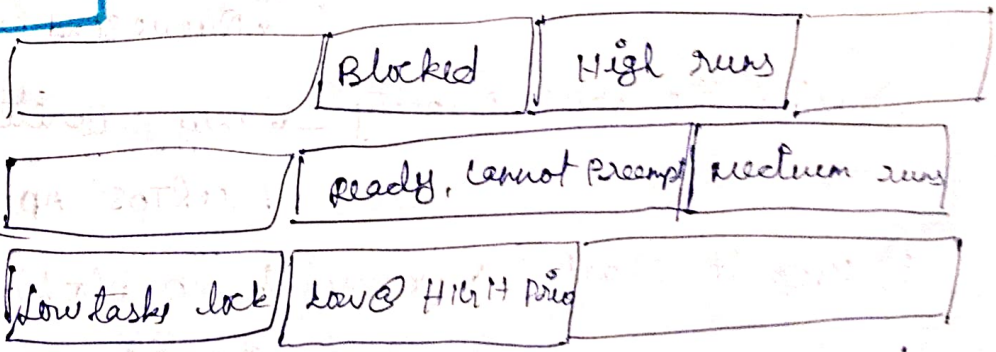
High (prio 3) ——→ Blocked : waits for lock — high runs

Med (" 2) · medium hogs the CPU

Low (prio 1) Low takes lock — Low gives

High needs the mutex held by low, but medium keeps preempting low. High effectively waits for medium

Priority Inheritance



when High blocks on the mutex, the kernel temporarily raises low to high's priority. Low finishes fast, gives the lock & drops back.

Task Notification & ISRs

The fastest way to wake a task

↳ Direct-to-task notifications
↳ Safe ISR rules & NVIC priority
↳ Deferred interrupt processing

Task Notifications: Light & Fast

1) Built into every task → Each task has a 32-bit notification value. No object to creat

2) Faster & lighter → uses no extra RAM; quicker. Like a binary semaphore.

3) Many roles → Binary or counting semphore, event flag or a 32-bit mailbox

4) Limits → One receiving task only, & the Rx cannot be an ISR.

Golden rules for ISRs in FreeRTOS

1) use the FromISR APIs → × SemaphoreGiveFromISR,
✓ TaskNotifyGiveFromISR,
× QueueSendFromISR.

2) Mind the NVIC priority → only priorities 5-15 may call FreeRTOS APIs

3) Keep it short → No printf, no blocking, no long loops inside an ISR

4) Yield when needed → End with portYIELD_FROM_ISR for an instant switch

Deferred interrupt processing

Hardware → ISR (short) → notify → Context switch
UART byte arrives → Reads data, clears flag → Give from ISR → handler task preempts
↓
Handler task
Parses & Process

ISR = urgent, tiny

Do only what the h/w needs: grab the data, clear the flag, notify

Task = heavy thing

parsing, Printf, file I/O & delays all belong in a task with the right priority.